Detecting Merged Git Branches

https://gemini.google.com/app/1330370b27dc5404

User prompt: I heard about a new git command that is capable of telling if a branch is merged. I think it is a recent publish. could you tell me more about it?

Response: What you likely heard about is either the addition to **`git for-each-ref`** introduced in **Git 2.47** (which solved the long-standing heuristic problem of detecting what base branch a branch or commit was created from/merged into), or the classic **`git branch --merged`** flags that frequently get highlighted in release and workflow guides.

1. The Recent Addition: `git for-each-ref --format="%(is-base:...)"` (Git 2.47+)

Historically, Git has had difficulty telling *where* a branch originally came from or what branch it was intended to merge into once development diverged.

In Git 2.47, Git added the **`%(is-base:<commit>)`** formatting atom to `git for-each-ref`. This atom dynamically evaluates the commit topology across candidate branches to pinpoint the exact base/parent branch a commit or feature branch belongs to:

```
# Find which branch acts as the base for your current branch
git for-each-ref \
  --format="%(refname:short) %(is-base:HEAD)" \
  refs/heads/
```

If a candidate branch contains the base commit, Git outputs the commit hash next to that branch's name, letting CLI tools and scripts identify whether your branch's history has already converged into a target mainline.

2. Standard Native Git Commands to Check Merged Status

If you are looking for the everyday command line ways to check if a specific branch is merged, Git provides several dedicated methods:

A. `git branch --merged`

To check which branches are merged into your current branch (e.g., when you are on `main`):

```
git branch --merged
```

Any branch appearing in this output has all of its commits already present in your current branch.

To check against a specific target without switching branches:

```
# List local branches merged into main
git branch --merged main

# Check if a specific feature branch is merged into main
git branch --merged main | grep "feature-branch"
```

B. Script-Friendly Ancestor Check: `git merge-base --is-ancestor`

If you need an exit code for CI/CD or shell scripts:

```
git merge-base --is-ancestor <feature-branch> <target-branch>
```

- **Exit code `0`**: The commits from `<feature-branch>` are completely contained in `<target-branch>` (effectively merged).
- **Exit code `1`**: Not merged.

C. `git log` Reachability Check

```
# If this returns no output, branch-a is fully contained within branch-b
git log <target-branch>..<feature-branch> --oneline
```
